



Draw It or Lose It
CS 230 Project Software Design Template
Version 3.0

Table of Contents

CS 230 Project Software Design Template	1
Table of Contents	2
Document Revision History	2
Executive Summary	3
Requirements	3
Design Constraints	3
System Architecture View	3
Domain Model	3
Evaluation	4
Recommendations	5

Document Revision History

Version	Date	Author	Comments
1.0	05/23/26	Joseph Hopkins	Initial draft; completed Executive Summary, Design Constraints, and Domain Model sections.
2.0	06/07/26	Joseph Hopkins	Project Two: completed the Evaluation table (Server Side, Client Side, and Development Tools) for Mac, Linux, Windows, and mobile platforms.
3.0	06/21/26	Joseph Hopkins	Project Three: completed the Recommendations section (Operating Platform, Operating Systems Architectures, Storage Management, Memory Management, Distributed Systems and Networks, and Security).

Instructions

Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

Creative Technology Solutions (CTS) has been engaged by The Gaming Room to design and develop a web-based, multi-platform version of their existing Android game, Draw It or Lose It. The current Android-only application limits the client's audience to a single platform. The Gaming Room wants to reach players on Linux, Mac, Windows, and mobile devices through a single, centrally hosted web application.

The proposed solution is a Java-based server-side application built on a layered, object-oriented domain model. The architecture uses two established software design patterns to meet all four of the client's stated requirements. The Singleton pattern, applied to the GameService class, ensures that only one instance of the game management service exists in memory at any given time, giving every part of the application a single, authoritative source of game state. The Iterator pattern, applied in the addGame(), addTeam(), and addPlayer() methods, traverses active entity collections to enforce name uniqueness before any new game, team, or player instance is created.

The class hierarchy (Entity as the abstract base, extended by Game, Team, and Player) reflects the natural structure of the game and was designed for readability, maintainability, and future scalability. Before development proceeds, The Gaming Room should be aware of three critical items: (1) a Java-capable web application server (such as Apache Tomcat or a cloud-hosted equivalent) is required for deployment; (2) game state currently resides in server memory and a database persistence layer will be needed before the application can support a production user base; and (3) a full platform evaluation and server recommendation are provided in the Evaluation and Recommendations sections of this document.

Requirements

Business Requirements

BR-1: The game application must support one or more competing teams per game session.

BR-2: Each team must be able to have multiple players assigned to it.

BR-3: Game and team names must be unique so that users can verify name availability before registering.

BR-4: Only one instance of the game may exist in memory at any given time to prevent conflicting game states.

Technical Requirements

TR-1: The application must be accessible via web browser across Linux, Mac, Windows, and mobile platforms without requiring platform-specific client software.

TR-2: The GameService class must implement the Singleton design pattern to enforce the single-instance constraint at the application level.

TR-3: Name-uniqueness checks in addGame(), addTeam(), and addPlayer() must use the Iterator design pattern to traverse active entity collections before creating any new instance.

TR-4: All entity classes (Game, Team, Player) must inherit from a common abstract base class (Entity) that encapsulates the shared id and name attributes.

Design Constraints

Transitioning Draw It or Lose It from a closed Android application to a web-based, distributed environment introduces the following design constraints.

1. Platform Independence. The application must run without modification on Linux, Mac, and Windows server environments and must be reachable from mobile browsers. Java on the JVM satisfies this requirement because bytecode is platform-neutral, but the deployment environment (OS, JVM version, web container) must be standardized to eliminate platform-specific runtime differences.

2. Stateless HTTP Protocol. Web browsers communicate over HTTP/HTTPS, a stateless protocol that carries no memory of previous requests. Unlike the Android app, where game state persists naturally in the running process, the web application must manage and restore game state on the server for every request. This requires either server-side session management or a persistent data store, adding complexity to the application architecture.

3. Single-Instance Constraint. The client requires that only one game exist in memory at a time. This is enforced by the Singleton pattern in `GameService`. In a single-server deployment this constraint is straightforward to satisfy, but if the application is later scaled horizontally across multiple servers, ensuring a globally single instance would require a shared data store or a distributed coordination mechanism.

4. Name Uniqueness and Concurrency. Game and team names must be unique across the system. The Iterator pattern enforces this at runtime by scanning active collections before any new entity is created. However, in a multi-user web environment, two players could simultaneously submit the same name and both pass the uniqueness check before either entry is saved. Production code must protect these critical sections with thread synchronization or database-level uniqueness constraints to prevent race conditions.

5. Bandwidth and Real-Time Rendering. The game renders stock drawings progressively over 30 seconds with strict timing requirements (fully rendered at the 30-second mark, one-guess opportunity at 15 seconds). Delivering image assets reliably over varying network connections—especially on cellular mobile devices—requires efficient content delivery, such as a content delivery network (CDN) and compressed image formats, to prevent rendering latency from breaking the game timer.

6. Security. Unlike the self-contained Android app, a public web application is exposed to the internet and must be hardened against common threats. At minimum, all traffic must be encrypted via HTTPS, user authentication tokens must be managed securely, and all inputs must be validated server-side to prevent injection attacks. These security requirements influence both the application design and the choice of hosting platform.

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

The UML class diagram for Draw It or Lose It illustrates a well-structured, object-oriented domain model built around six classes. The following describes each class, how the classes relate to one another, and the OOP principles that each relationship demonstrates.

Entity (abstract base class). Entity is an abstract base class that holds two attributes shared by every persistent object in the system: a long id (a unique numeric identifier) and a String name (a display label). It provides public getter methods (getId() and getName()) but no setters, making every entity immutable after construction. Placing these shared members in Entity is an application of inheritance and the DRY (Don't Repeat Yourself) principle: instead of declaring id and name in Game, Team, and Player separately, they are declared once and inherited by all three subclasses, eliminating code duplication and ensuring consistent behavior.

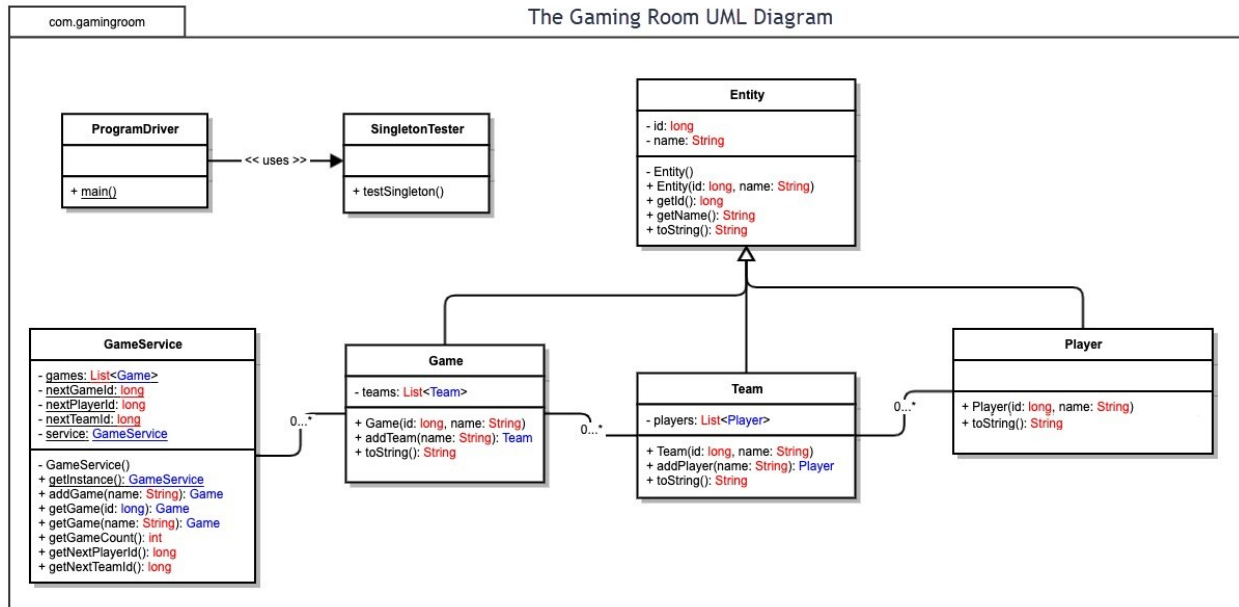
Game (extends Entity). Game inherits id and name from Entity and adds a private List<Team> that it manages internally. The relationship between GameService and Game is a one-to-many association: GameService holds a List<Game> and a Game can contain zero or more Team instances. This layered containment is an example of object composition. The addTeam() method uses the Iterator pattern to scan existing teams before creating a new one, directly satisfying the requirement that team names be unique within a game.

Team (extends Entity). Team inherits id and name from Entity and adds a private List<Player>. Like Game, Team uses the Iterator pattern in addPlayer() to enforce player-name uniqueness within the team. The one-to-many relationship between Team and Player (a team contains zero or more players) mirrors the Game-to-Team relationship, creating a consistent three-level hierarchy: Game > Team > Player.

Player (extends Entity). Player is the simplest concrete class in the hierarchy. It inherits id and name from Entity and adds no additional state, which is a direct application of encapsulation: all observable player data is accessed through the inherited getters, and the internal fields remain private and unmodifiable.

GameService (Singleton). GameService sits outside the Entity inheritance hierarchy and acts as the entry point and coordinator for the entire domain model. Its private constructor and static getInstance() method implement the Singleton pattern, guaranteeing that only one GameService object exists in memory. The class also holds the global id counters (nextGameId, nextTeamId, and nextPlayerId) that assign unique identifiers to every entity in the system, satisfying the requirement that every game, team, and player instance have a unique identifier. ProgramDriver and SingletonTester both use GameService, demonstrating through the testSingleton() method that multiple callers share the same object.

Taken together, the diagram demonstrates all four core OOP principles: abstraction (Entity defines a common interface for all entities), encapsulation (private fields exposed only through getters, no setters), inheritance (Game, Team, and Player all extend Entity, reusing shared behavior), and polymorphism (any subclass instance can be treated as an Entity wherever the base type is expected). The Iterator and Singleton design patterns build on these principles to address the specific constraints of the application: unique identifiers, name-uniqueness enforcement, and controlled single-instance access.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	macOS is a UNIX-based OS that runs the JVM reliably, making it a viable development environment. Apple hardware is high-quality and the OS is stable, but licensing costs are significant (Apple hardware required). macOS itself carries no separate server-OS license fee, but it is legally tied to Apple hardware and is not offered as a managed hosting option by the major cloud providers (AWS, Google Cloud, Azure). Choosing it as a server would therefore mean buying and self-managing Apple hardware, which is why macOS is not a typical production web hosting choice and is best used as a developer workstation rather than a deployment target.	Linux is the dominant platform for web server hosting. It is open-source, has minimal licensing cost, offers excellent JVM support, and is the native environment for most cloud platforms (AWS, GCP, Azure). It is highly stable and well-suited for running Java application servers such as Apache Tomcat. The main weakness is a steeper learning curve for administrators unfamiliar with the command line, but this is easily addressed by choosing a managed cloud deployment.	Windows Server supports the JVM and can host Java web applications, and it offers strong tooling integration for developers who use Visual Studio or IIS. However, Windows Server is a paid, proprietary operating system licensed per processor core (with separate client access licenses), and cloud providers charge a per-hour premium for Windows instances over equivalent Linux instances. This makes it the most expensive of the three server options for this project. Windows is also less commonly used for Java-based web hosting than Linux, and its memory and resource overhead tends to be higher than a minimal Linux installation, which can further affect performance and cloud hosting costs.	Mobile devices (iOS and Android phones and tablets) are client-side endpoints, not appropriate hosting platforms for a web server. They are, however, a primary target audience for the web client. Battery constraints, variable network conditions, and diverse screen sizes all affect client-side rendering. The web-based architecture addresses this by offloading all game logic to the server; mobile devices only need a browser to participate.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform:** Linux (specifically a cloud-hosted Linux environment such as AWS EC2, Google Cloud, or Azure) is the recommended operating platform. Linux is open-source, has no licensing fees, offers first-class JVM support, and is natively supported by every major cloud provider. Hosting the application in the cloud means it is immediately reachable from any client platform (Mac, Windows, mobile) without additional client software installation.
2. **Operating Systems Architectures:** The foundation is a Linux server in the cloud (for example, an AWS EC2 instance running Ubuntu Server) that runs the Java application server Apache Tomcat. Linux is a stable, UNIX-based operating system that can run many users and programs at once while keeping them separated, which makes it a dependable host for a game service that stays running around the clock. The application is organized in three tiers: the player's web browser, the Java server that holds the game logic, and a database (such as PostgreSQL) that stores the saved game data. Secure HTTPS connections are handled by a reverse proxy (NGINX) placed in front of Tomcat.
3. **Storage Management:** Draw It or Lose It needs to store a few different kinds of data, and each kind fits a different place. The game records (games, teams, players, and scores) belong in a database such as PostgreSQL, which keeps the data accurate when many people play at once and can be rented as a managed service (AWS RDS or Google Cloud SQL) so the team does not have to run the database server itself. The image library is different: it is a large collection of picture files, so it is better kept in dedicated image storage (such as AWS S3) than inside the database. The most important decision is to keep the roughly 1.6 GB library (about 200 images at 8 MB each) in one place on the server rather than copying it into every client, which would make a mobile install far too large and updates difficult (Bashir, 2019). A content delivery network (CDN) then keeps copies of the images on servers near the players for faster loading, and smaller versions of each image can be sent to phones so they download only what their screen needs.
4. **Memory Management:** Because each clue must be drawn within a strict thirty-second window, memory is the main resource to manage carefully. The server should not load the whole 1.6 GB image library into memory at once. Instead, it should load each image only when it is needed and quietly load the next puzzle's image while the current one is on screen, so the drawing never pauses (Silberschatz et al., 2008). When a round ends, the image is released so its memory can be reused for the next one. Java handles this cleanup automatically through a process called garbage collection, so developers do not have to free memory by hand, and the Singleton design keeps just one shared game service in memory instead of many copies. The server should also send phones a smaller version of each image rather than the full 8 MB file, because mobile devices have less memory to work with.
5. **Distributed Systems and Networks:** The recommended architecture is a client-server model where the Java application server is the single authoritative source of game state and all clients (Mac browsers, Windows browsers, Android phones, iPhones) communicate with it over HTTPS. RESTful HTTP endpoints allow the client UI to query game state and submit player actions. Real-

time drawing rendering can be delivered via WebSockets or HTTP Server-Sent Events, which push image data from the server to all connected clients simultaneously. Key network dependencies fall into three areas, each with a mitigation. First, the availability of the cloud host, which is addressed by using a managed platform with a service-level agreement (SLA). Second, the reliability of the CDN serving game images, which is addressed by caching assets at multiple edge locations. Third, variable client network conditions, which are addressed by compressing images and running the game timer server-side, so a brief connectivity blip does not desynchronize the game state.

6. **Security:** All traffic between clients and the server must be encrypted using TLS 1.2 or higher (HTTPS), preventing eavesdropping on player names and game data. User sessions should be managed with short-lived, server-signed tokens (JWT or secure cookies with HttpOnly and SameSite flags) to prevent session hijacking. Input validation must be enforced server-side on all incoming game and team names, guarding against injection attacks. On Linux, the server process should run as a least-privilege user (not root), and OS-level firewalls (iptables or cloud security groups) should restrict inbound traffic to HTTPS only. At the application layer, the Iterator-based uniqueness checks also serve as an implicit guard against duplicate-entry exploits. Regular dependency updates and security scanning (OWASP Dependency-Check) should be included in the CI/CD pipeline to address known Java library vulnerabilities over time.

References

Bashir, F. (2019, July 5). *What is serverless architecture? What are its pros and cons?* freeCodeCamp.
<https://www.freecodecamp.org/news/what-is-serverless-architecture-what-are-its-pros-and-cons/>

Silberschatz, A., Galvin, P. B., & Gagne, G. (2008). *Operating system concepts* (8th ed.). John Wiley & Sons.